

]SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName:B. Tech		Assignment Type: Lab	AcademicYear:2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
NS_2 (Mounika)			
rCourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week5- Wednesday	Time(s)	
Duration	2 Hours	Applicableto Batches	
AssignmentNumber:9.3(Present assignment number)/24(Total number of assignments)			
NAME: VADDEM CHANDANA HALL TICKET NO: 2403A51186 BATCH-09			
Q.No.	Question	ExpectedTime to complete	
1	Lab 8: Documentation Generation: Automatic documentation and code comments Lab Objectives:	Week4 - Wednesday	

	• To understand the importance of documentation and code comments in software development. • To explore how AI-assisted coding tools can generate meaningful documentation and inline comments. • To practice generating function-level and module-level docstrings automatically. • To evaluate the quality, accuracy, and limitations of AI-generated documentation. • To develop a small automated tool for documentation generation in Python.. Lab Outcomes (LOs): After completing this lab, students will be able to: • Apply AI-assisted coding tools to generate docstrings and inline comments for Python code. • Critically analyze AI-generated documentation for correctness, completeness, and readability. • Create structured documentation (function-level, module-level) following standard formats. • Design and implement a mini documentation generator tool to automate code commenting and docstring creation. Task Description#1 Basic Docstring Generation • Write python function to return sum of even and odd numbers in the given list. • Incorporate manual docstring in code with Google Style • Use an AI-assisted tool (e.g., Copilot, Cursor AI) to generate a docstring describing the function. • Compare the AI-generated docstring with your manually written one. Expected Outcome#1: Students understand how AI can produce function-level documentation Manually :	
--	--	--

```

9.3.2.py > sum_even_odd
1  def sum_even_odd(numbers):
2      """
3      Returns the sum of even and odd numbers in the given list.
4
5      Parameters
6      -----
7      numbers : list of int
8      |         List of integers.
9
10     Returns
11     -----
12     tuple
13     |     A tuple containing the sum of even numbers and the sum of odd numbers.
14     """
15     # Initialize variables to store sums
16     sum_even = 0
17     sum_odd = 0
18     # Loop through each number in the list
19     for num in numbers:
20         if num % 2 == 0:
21             sum_even += num # Add to even sum if number is even
22         else:
23             sum_odd += num # Add to odd sum if number is odd
24     return sum_even, sum_odd # Return both sums as a tuple
25
26 # Example usage
27 numbers = [1, 2, 3, 4, 5, 6] # List of numbers to process
28 even_sum, odd_sum = sum_even_odd(numbers) # Get sums of even and odd numbers
29 print(f"Sum of even numbers: {even_sum}") # Print sum of even numbers
30 print(f"Sum of odd numbers: {odd_sum}") # Print sum of odd numbers

```

AI Generated:

```

9.3.2.py > ...
1  def sum_even_odd(numbers):
2      """
3      Calculates the sum of even and odd numbers in a list.
4
5      Args:
6      |     numbers (list): A list of integers.
7
8      Returns:
9      |     tuple: A tuple containing the sum of even numbers and the sum of odd numbers.
10     """
11     sum_even = 0
12     sum_odd = 0
13     for num in numbers:
14         if num % 2 == 0:
15             sum_even += num
16         else:
17             sum_odd += num
18     return sum_even, sum_odd
19
20 # Example usages
21 numbers = [1, 2, 3, 4, 5, 6] # List of numbers to process
22 even_sum, odd_sum = sum_even_odd(numbers) # Get sums of even and odd numbers
23 print(f"Sum of even numbers: {even_sum}") # Print sum of even numbers
24 print(f"Sum of odd numbers: {odd_sum}") # Print sum of odd numbers

```

Output:

```

PS C:\Users\SRIJA\Desktop\2403a51194> & C:/Users/SRIJA/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/SRIJA/Desktop/2403a51194/9.3.2.py
Sum of even numbers: 12
Sum of odd numbers: 9
PS C:\Users\SRIJA\Desktop\2403a51194>

```

Task Description#2 Automatic Inline Comments

- Write python program for **sru_student** class with attributes like name, roll no., hostel_status and **fee_update** method and **display_details** method.
- Write comments manually for each line/code block
- Ask an AI tool to add inline comments explaining each line/step.
- Compare the AI-generated comments with your manually written one.

Expected Output#2: Students critically analyze AI-generated code comments.

Manually:

```
9.3.2.1.py > sru_student > fee_update
1 class sru_student:
2     def __init__(self, name, roll_no, hostel_status):
3         # Assign the provided name to the student
4         self.name = name
5         # Assign the provided roll number to the student
6         self.roll_no = roll_no
7         # Assign the provided hostel status (Yes/No) to the student
8         self.hostel_status = hostel_status
9         # Initialize the fee payment status as unpaid (False)
10        self.fee_paid = False
11
12    def fee_update(self, status):
13        # Update the fee payment status to the given value (True or False)
14        self.fee_paid = status
15
16    def display_details(self):
17        # Display the student's name
18        print(f"Name: {self.name}")
19        # Display the student's roll number
20        print(f"Roll No: {self.roll_no}")
21        # Display the student's hostel status
22        print(f"Hostel Status: {self.hostel_status}")
23        # Display the student's fee payment status
24        print(f"Fee Paid: {self.fee_paid}")
25
26    # Example usage
27    student1 = sru_student("Anil", "2403A51194", "Yes") # Create a student object with name, roll number, and hostel status
28    student1.fee_update(True) # Mark the student's fee as paid
29    student1.display_details() # Print all details of the student
```

AI Generated:

```
9.3.2.1.py > ...
1 class sru_student:
2     def __init__(self, name, roll_no, hostel_status):
3         # AI: Store the student's name
4         self.name = name
5         # AI: Store the student's roll number
6         self.roll_no = roll_no
7         # AI: Store the student's hostel status (Yes/No)
8         self.hostel_status = hostel_status
9         # AI: Initialize the fee payment status as unpaid (False)
10        self.fee_paid = False
11
12    def fee_update(self, status):
13        # AI: Update the fee payment status to True (paid) or False (unpaid)
14        self.fee_paid = status
15
16    def display_details(self):
17        # AI: Print the student's name
18        print(f"Name: {self.name}")
19        # AI: Print the student's roll number
20        print(f"Roll No: {self.roll_no}")
21        # AI: Print the student's hostel status
22        print(f"Hostel Status: {self.hostel_status}")
23        # AI: Print the student's fee payment status
24        print(f"Fee Paid: {self.fee_paid}")
25
26    # Example usage
27    student1 = sru_student("sreeja", "2403A51194", "Yes") # AI: Create a student object with name, roll number, and hostel status
28    student1.fee_update(True) # AI: Mark the student's fee as paid
29    student1.display_details() # AI: Print all details of the student
```

Output:

```
Name: sreeja
Roll No: 2403A51194
Hostel Status: Yes
Fee Paid: True
PS C:\Users\SRIJA\Desktop\2403a51194>
```

Task Description#3

- Write a Python script with 3–4 functions (e.g., calculator: add, subtract, multiply, divide).
- Incorporate manual **docstring** in code with NumPy Style
- Use AI assistance to generate a module-level docstring + individual function docstrings.
- Compare the AI-generated docstring with your manually written one.

Expected Output#3: Students learn structured documentation for multi-function scripts

Manually:

```
9.3.2.py > ...
1  """
2  calculator.py
3
4  A simple calculator module that provides basic arithmetic operations.
5
6  This module includes functions to add, subtract, multiply, and divide two numbers.
7  Each function takes two numeric arguments and returns the result of the operation.
8  Division by zero raises a ValueError.
9
10 Functions
11 -----
12 add(a, b)
13     Returns the sum of two numbers.
14 subtract(a, b)
15     Returns the difference between two numbers.
16 multiply(a, b)
17     Returns the product of two numbers.
18 divide(a, b)
19     Returns the quotient of two numbers.
20 """
21
22 def add(a, b):
23     """
24     Add two numbers and return the result.
25
26     Parameters
27     -----
28     a : float or int
29         First number.
30     b : float or int
31         Second number.
32
33     Returns
34     -----
35     float or int
36         The sum of a and b.
37     """
```

```

25 def subtract(a, b):
37     """
38     float or int
39     | The difference of a and b.
40     """
41     return a - b
42
43 def multiply(a, b):
44     """
45     Multiply two numbers.
46
47     Parameters
48     -----
49     a : float or int
50     | First number.
51     b : float or int
52     | Second number.
53
54     Returns
55     -----
56     float or int
57     | The product of a and b.
58     """
59     return a * b
60
61 def divide(a, b):
62     """
63     Divide one number by another.
64
65     Parameters
66     -----
67     a : float or int
68     | Numerator.
69     b : float or int
70     | Denominator.
71
72     Returns

```

```

73     -----
74     float
75     |     The quotient of a and b.
76
77     Raises
78     -----
79     ValueError
80     |     If b is zero.
81     """
82     if b == 0:
83         raise ValueError("Cannot divide by zero.")
84     return a / b

```

AI Generated:

```

9.3.2.py > ...
1  """
2  calculator.py
3
4  A simple calculator module that provides basic arithmetic operations.
5
6  This module includes functions to add, subtract, multiply, and divide two numbers.
7  Each function takes two numeric arguments and returns the result of the operation.
8  Division by zero raises a ValueError.
9
10 Functions
11 -----
12 add(a, b)
13 |     Returns the sum of two numbers.
14 subtract(a, b)
15 |     Returns the difference between two numbers.
16 multiply(a, b)
17 |     Returns the product of two numbers.
18 divide(a, b)
19 |     Returns the quotient of two numbers.
20 """
21
22 def add(a, b):
23     """
24     Add two numbers and return the result.
25
26     Parameters
27     -----
28     a : float or int
29     |     First number.
30     b : float or int
31     |     Second number.
32
33     Returns
34     -----
35     float or int
36     |     The sum of a and b.
37     """

```

```
22 def add(a, b):
38     return a + b
39
40 def subtract(a, b):
41     """
42     Subtract b from a and return the result.
43
44     Parameters
45     -----
46     a : float or int
47         |   Number to subtract from.
48     b : float or int
49         |   Number to subtract.
50
51     Returns
52     -----
53     float or int
54         |   The difference of a and b.
55     """
56     return a - b
57
58 def multiply(a, b):
59     """
60     Multiply two numbers and return the result.
61
62     Parameters
63     -----
64     a : float or int
65         |   First number.
66     b : float or int
67         |   Second number.
68
69     Returns
70     -----
71     float or int
72         |   The product of a and b.
73     """
```

```

58 def multiply(a, b):
71     float or int
72     |
72     The product of a and b.
73     """
74     return a * b
75
76 def divide(a, b):
77     """
78     Divide a by b and return the result.
79
80     Parameters
81     -----
82     a : float or int
83     |
83     Numerator.
84     b : float or int
85     |
85     Denominator.
86
87     Returns
88     -----
89     float
90     |
90     The quotient of a and b.
91
92     Raises
93     -----
94     ValueError
95     |
95     If b is zero.
96     """
97     if b == 0:
98         raise ValueError("Cannot divide by zero.")
99     return a / b

```

Output:

```

Sum of odd numbers: 9
Sum of odd numbers: 9
Sum of odd numbers: 9
Sum of odd numbers: 9
PS C:\Users\SRIJA\Desktop\2403a51194> & C:/Users/SRIJA/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Us
/9.3.2.py

```

Push documentation whole workspace as .md file in GitHub Repository

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots